

Fleet Tracking Project

Name: Priteesh Prabhav Mistry

Matricola: 568529

1. Introduction

The Problem: Today, logistics companies have thousands of trucks on the road. These trucks have IoT sensors that constantly send data like speed, fuel levels, and current location. The problem is that traditional SQL databases struggle to handle this massive, non-stop flow of information. If we try to put everything into one rigid table, the system will slow down or crash.

Project Objective: The goal of this project is to build a system that can collect all this sensor data in real-time using an MQTT server. Instead of forcing all the data into one place, I built a Python application that sorts the data based on its topic and saves it into three different types of databases: MySQL, MongoDB, and Neo4j. This way, we use the best database for each specific type of data.

2. Implementation of MQTT Server and Python

To make the devices talk to the databases, I used **Eclipse Mosquitto**. I chose Mosquitto because it is very lightweight, runs easily inside a Docker container, and is great at handling a lot of messages very quickly without using up too much computer memory.

I wrote the main application in Python using the **paho-mqtt** library. The Python script (the router) connects to the Mosquitto broker and subscribes to the data streams. When the simulated trucks publish their JSON data, the Python script catches it, reads the MQTT topic, and decides which database should receive it.

3. Database Integration and Motivation

I have used three completely different database technologies. Here is the reasoning for why I chose each one for specific tasks:

A. MySQL (SQL) for Static Admin Data: I used MySQL to store the vehicle registry (Vehicle ID, License Plate, and Status). I chose a relational database for this because this data does not change very often. A truck's license plate stays the same. SQL is perfect for this because it keeps the data organized in strict tables and prevents duplicate entries by using a Primary Key. The Python script routes data to MySQL only when it comes from the **fleet/admin/registry** topic.

B. MongoDB (Document NoSQL) for High-Velocity Telemetry: I used MongoDB to store the live sensor data (Speed, Fuel Level, and Timestamps). The trucks generate this data every few

seconds. I chose MongoDB because it is a Document database that naturally stores JSON data. Unlike SQL, MongoDB doesn't need a strict table structure, so it can easily catch and save thousands of unstructured sensor readings very quickly without slowing down. This data comes from the **fleet/telemetry/trucks** topic.

C. Neo4j (Graph NoSQL) for Spatial Tracking: I used Neo4j, using the py2neo library, to map where the trucks are actually going. Every time a truck sends a location update, Neo4j creates a "Node" for the Truck, a "Node" for the City, and draws a "VISITED" relationship line between them. I chose a Graph database because if a manager wants to see all the trucks that passed through a specific city, a graph database can show that relationship instantly, which is much faster than trying to write a complicated JOIN query in SQL.

4. Test and Performance Evaluation

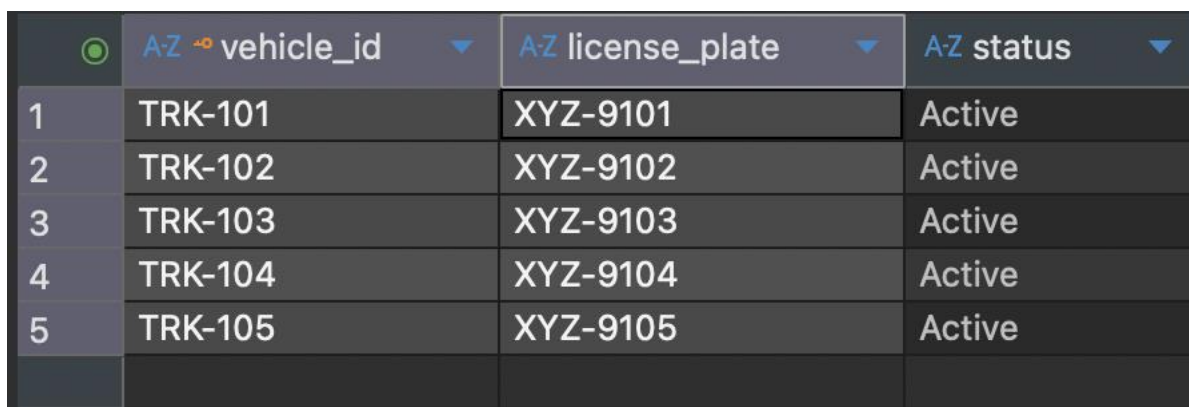
To make sure the system actually works under pressure, I ran a performance stress test.

Normally, the simulated trucks send data every 2 seconds. To test the system's reliability, I changed the Python code to send data every 0.1 seconds. This means the Mosquitto broker and the databases were suddenly getting spammed with messages extremely fast.

Results: The system handled the load perfectly. The Python terminal printed the routing logs very quickly, but the script did not crash, and my computer did not freeze. MongoDB was able to ingest the fast stream of JSON data without dropping any messages, proving that this architecture is highly scalable and can handle real-world big data speeds.

5. Visual Verification of Databases

Below are the screenshots showing the data successfully stored in all three databases at the same time:



	A-Z vehicle_id	A-Z license_plate	A-Z status
1	TRK-101	XYZ-9101	Active
2	TRK-102	XYZ-9102	Active
3	TRK-103	XYZ-9103	Active
4	TRK-104	XYZ-9104	Active
5	TRK-105	XYZ-9105	Active

Figure 1: MySQL Vehicle Registry (This shows the static data stored in rows and columns)

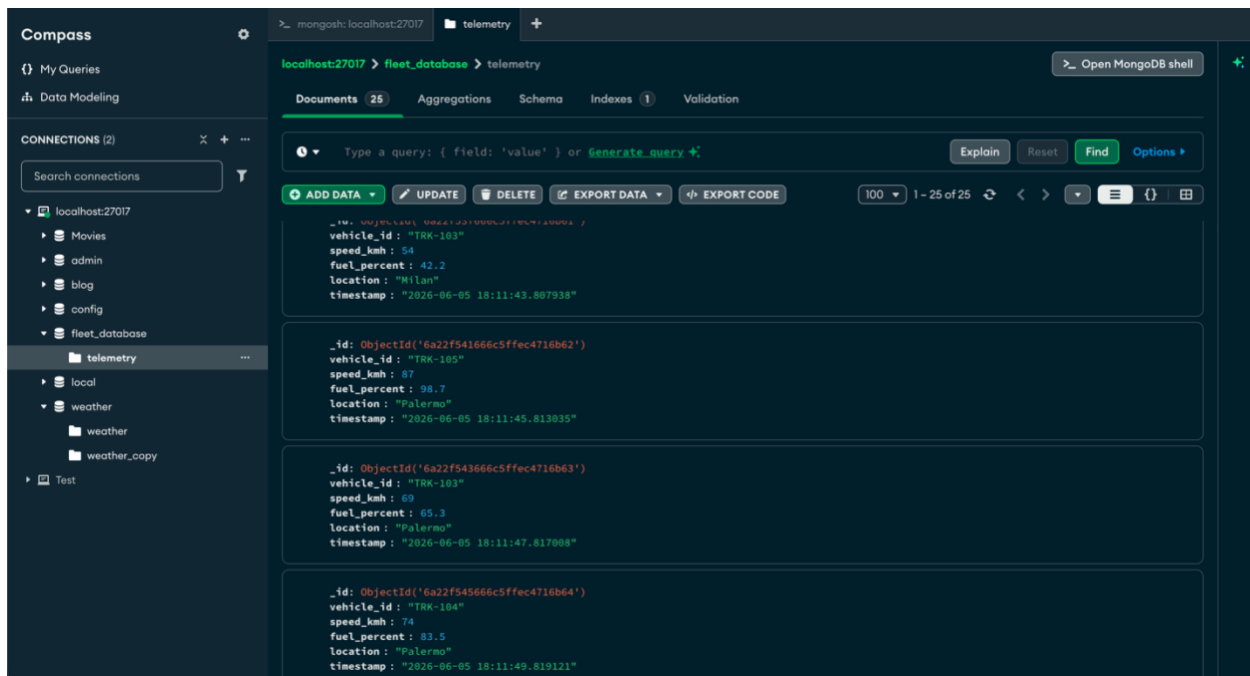


Figure 2: MongoDB Telemetry Documents (This shows the live JSON sensor data ingested without a strict schema)

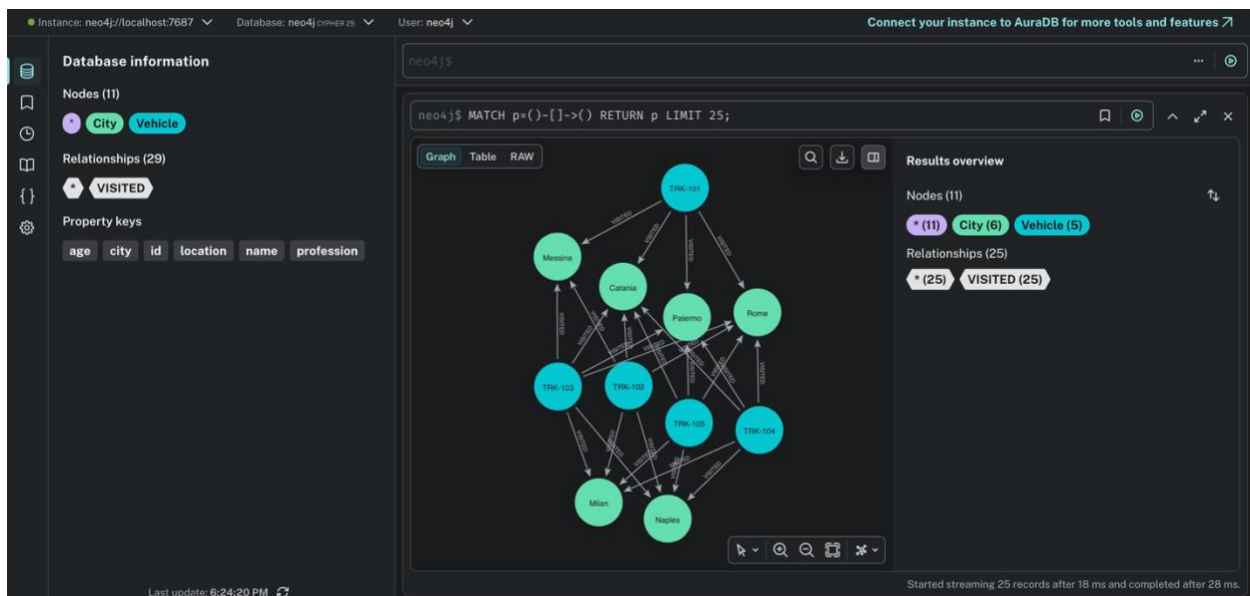


Figure 3: Neo4j Spatial Graph (This shows the physical network of vehicles connected to the cities they visited)

6. Conclusions and Future Developments

Summary: This project successfully proves that combining an MQTT server with Python is a great way to handle IoT big data. By routing data based on topics, we can use SQL for safe static data, MongoDB for fast unstructured data, and Neo4j for relationship tracking, all at the exact same time.

Future Developments: If I were to improve this system in the future, I would add a "Geofencing Alert System." Since Neo4j already maps the trucks to specific cities, I could write a Python script that constantly checks the graph. If a truck enters a city that it is not supposed to be in, the system could automatically publish an emergency alert message back to the MQTT broker to warn the fleet manager.